ВІННИЦЬКИЙ НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ

ІНСТИТУТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ ТА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

КАФЕДРА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

Хеш-функції та хеш-таблиці

Звіт про виконання лабораторної роботи № 5

Виконав (ла): ст. 2 курсу, групи 2ПІ-25Б

Коваль Олександр

Перевірив: викладач Карась О.В.

Тема: Дослідження хеш-функцій та хеш-таблиць. Хеш-функція SHABAL. Метод множення.

Мета: ознайомитися з відомими алгоритмами побудови хеш-функцій; реалізувати хеш-функцію SHABAL та власну хеш-функцію методом множення; застосувати їх для обчислення контрольної суми файлу і порівняти якість розподілу ключів у хеш-таблиці.

Вінниця 2026

ЗАВДАННЯ

1. Обчислити хеш-функцію відомим методом. Для варіанта 1.12 — хеш-функція SHABAL.

2. Придумати та реалізувати власну хеш-функцію. Варіант 12 парний, тому застосовано метод множення.

3. Застосувати розроблені хеш-функції для пункту 3б) — обчислення контрольної суми заданого файлу, яка дописується в кінець цього файлу.

4. Передбачити в програмі зручні методи роботи користувача (меню з вибором дії).

5. Порівняти хеш-функції: створити хеш-таблицю на 10^3–10^4 значень, заповнити її на 50–70 % і порахувати кількість колізій для 500–1000 ключів; врахувати вплив діапазону вхідних даних.

КОД ПРОГРАМИ

```
#include <iostream>

#include <fstream>

#include <vector>

#include <string>

#include <random>

#include <iomanip>

#include <unordered_set>

#include <cstdint>


using namespace std;









uint32_t rotl(uint32_t x, int n) {

    return (x << n) | (x >> (32 - n));

}
```

```
uint32_t U(uint32_t x) { return x * 3; }
uint32_t V(uint32_t x) { return x * 5; }



void decodeBlock(const unsigned char* buf, uint32_t M[16]) {
    for (int i = 0; i < 16; i++)
        M[i] = buf[4*i] | (buf[4*i+1] << 8) |
               (buf[4*i+2] << 16) | (uint32_t(buf[4*i+3]) << 24);
}







void shabalP(uint32_t A[], uint32_t B[], uint32_t C[], const uint32_t M[]) {
    for (int i = 0; i < 16; i++)
        B[i] = rotl(B[i], 17);

    for (int j = 0; j < 3; j++)
        for (int i = 0; i < 16; i++) {
            int k = (i + 4*j) % 12;
            A[k] = U(A[k] ^ C[(8 - i + 16) % 16] ^ V(rotl(A[(k + 11) % 12], 15)))
                   ^ M[i] ^ B[(i + 13) % 16] ^ (B[(i + 9) % 16] & ~B[(i + 6) % 16]);
            B[i] = ~(rotl(B[i], 1) ^ A[k]);
        }

    for (int j = 0; j < 36; j++)
        A[j % 12] += C[(j + 3) % 16];
}
```

```
void shabalBlock(uint32_t A[], uint32_t B[], uint32_t C[], const uint32_t M[],

                 uint32_t& Wlow, uint32_t& Whigh) {

    for (int i = 0; i < 16; i++) B[i] += M[i];

    A[0] ^= Wlow;

    A[1] ^= Whigh;

    shabalP(A, B, C, M);

    for (int i = 0; i < 16; i++) C[i] -= M[i];

    for (int i = 0; i < 16; i++) { uint32_t t = B[i]; B[i] = C[i]; C[i] = t; }

    if (++Wlow == 0) ++Whigh;

}




void shabal512(const vector<unsigned char>& msg, uint32_t out[16]) {

    uint32_t A[12] =
{0x20728DFDu,0x46C0BD53u,0xE782B699u,0x55304632u,0x71B4EF90u,0x0EA9E82Cu,


0xDBB930F1u,0xFAD06B8Bu,0xBE0CAE40u,0x8BD14410u,0x76D2ADACu,0x28ACAB7Fu};

    uint32_t B[16] =
{0xC1099CB7u,0x07B385F3u,0xE7442C26u,0xCC8AD640u,0xEB6F56C7u,0x1EA81AA9u,


0x73B9D314u,0x1DE85D08u,0x48910A5Au,0x893B22DBu,0xC5A0DF44u,0xBBC4324Eu,

                     0x72D2F240u,0x75941D99u,0x6D8BDE82u,0xA1A7502Bu};

    uint32_t C[16] =
{0xD9BF68D1u,0x58BAD750u,0x56028CB2u,0x8134F359u,0xB5D469D8u,0x941A8CC2u,


0x418B2A6Eu,0x04052780u,0x7F07D787u,0x5194358Fu,0x3C60D665u,0xBE97D79Au,

                     0x950C3434u,0xAED9A06Du,0x2537DC8Du,0x7CDB5969u};

    uint32_t Wlow = 1, Whigh = 0;

    uint32_t M[16];



    size_t pos = 0;
```

```
    while (msg.size() - pos >= 64) {
        decodeBlock(&msg[pos], M);
        shabalBlock(A, B, C, M, Wlow, Whigh);
        pos += 64;
    }

    unsigned char buf[64] = {};
    for (size_t i = 0; pos + i < msg.size(); i++) buf[i] = msg[pos + i];
    buf[msg.size() - pos] = 0x80;
    decodeBlock(buf, M);

    for (int i = 0; i < 16; i++) B[i] += M[i];
    A[0] ^= Wlow;
    A[1] ^= Whigh;
    shabalP(A, B, C, M);

    for (int i = 0; i < 3; i++) {
        for (int k = 0; k < 16; k++) { uint32_t t = B[k]; B[k] = C[k]; C[k] = t; }
        A[0] ^= Wlow;
        A[1] ^= Whigh;
        shabalP(A, B, C, M);
    }
    for (int i = 0; i < 16; i++) out[i] = B[i];
}
```

```
uint32_t myHash(const unsigned char* data, size_t len) {
    const uint32_t A = 0x9E3779B9u;
    uint32_t h = 0x811C9DC5u;
    for (size_t i = 0; i < len; i++)
        h = h * A + data[i] + 1;
    return h;
}



uint32_t divHash(uint32_t h, uint32_t m) {
    return h % m;
}









vector<unsigned char> readFile(const string& name, bool& ok) {
    ifstream f(name, ios::binary);
    ok = (bool)f;
    if (!ok) return {};
    vector<unsigned char> d;
    char buf[4096];
    while (f.read(buf, 4096) || f.gcount())
        d.insert(d.end(), buf, buf + f.gcount());
    return d;
}
```

```
void calcChecksum(const string& name) {
    bool ok;
    vector<unsigned char> data = readFile(name, ok);
    if (!ok) { cout << "Не вдалося відкрити файл: " << name << endl; return; }

    uint32_t h[16];
    shabal512(data, h);
    uint32_t mine = myHash(data.data(), data.size());

    cout << "Файл: " << name << "   (" << data.size() << " байт)" << endl;
    cout << "SHABAL-512 (512 біт):" << endl << "  " << hex << setfill('0');
    for (int i = 0; i < 16; i++) cout << setw(8) << h[i];
    cout << dec << setfill(' ') << endl;
    cout << "Власна, метод множення (32 біт): " << hex << setfill('0')
         << setw(8) << mine << dec << setfill(' ') << endl;
    cout << "Той самий хеш за методом дішення: " << divHash(mine, 8192) << endl;
}


void appendChecksum(const string& name) {
    bool ok;
    vector<unsigned char> data = readFile(name, ok);
    if (!ok) { cout << "Не вдалося відкрити файл: " << name << endl; return; }

    uint32_t h[16];
    shabal512(data, h);

    unsigned char sum[64];
    for (int i = 0; i < 16; i++)
```

```
        for (int j = 0; j < 4; j++)
            sum[4*i + j] = (unsigned char)(h[i] >> (8*j));

    ofstream f(name, ios::binary | ios::app);
    f.write((char*)sum, 64);
    f.close();
    cout << "Контрольну суму 64 байти дописано в кінець файлу " << name << endl;
}



void verifyChecksum(const string& name) {
    bool ok;
    vector<unsigned char> data = readFile(name, ok);
    if (!ok) { cout << "Не вдалося відкрити файл: " << name << endl; return; }
    if (data.size() < 64) { cout << "Файл закороткий, суми в ньому немає." << endl; return;
}

    size_t n = data.size() - 64;
    vector<unsigned char> body(data.begin(), data.begin() + n);

    uint32_t h[16];
    shabal512(body, h);

    bool same = true;
    for (int i = 0; i < 16; i++) {
        uint32_t saved = data[n + 4*i] | (uint32_t(data[n + 4*i+1]) << 8) |
                         (uint32_t(data[n + 4*i+2]) << 16) |
                         (uint32_t(data[n + 4*i+3]) << 24);
        if (saved != h[i]) same = false;
    }
```

```
    cout << (same ? "Перевірка УСПІШНА — контрольна сума збігається."
                  : "Перевірка НЕВДАЛА — файл було змінено!") << endl;
}




vector<uint32_t> makeKeys(uint32_t range, int count, mt19937& gen) {
    vector<uint32_t> keys;
    unordered_set<uint32_t> used;
    uniform_int_distribution<uint32_t> dist(0, range - 1);
    while ((int)keys.size() < count) {
        uint32_t k = dist(gen);
        if (used.insert(k).second) keys.push_back(k);
    }
    return keys;
}



uint32_t shabal512of(uint32_t key) {
    unsigned char b[4] = {(unsigned char)key, (unsigned char)(key >> 8),
                          (unsigned char)(key >> 16), (unsigned char)(key >> 24)};
    uint32_t h[16];
    shabal512(vector<unsigned char>(b, b + 4), h);
    return h[0];
}



uint32_t myHashOf(uint32_t key) {
    unsigned char b[4] = {(unsigned char)key, (unsigned char)(key >> 8),
```

```
                        (unsigned char)(key >> 16), (unsigned char)(key >> 24)};
    return myHash(b, 4);
}



void collisions() {
    const int bits = 13;
    const int m = 1 << bits;
    const int fill = m * 60 / 100;
    const int keys = 700;

    const char* names[3] = {"SHABAL-512", "Множення", "Дішення"};
    uint32_t ranges[3] = {10000u, 1000000u, 1000000000u};
    const char* rname[3] = {"0..10^4", "0..10^6", "0..10^9"};

    cout << "Розмір таблиці " << m << " комірок, заповнення " << fill
         << " (" << fill*100/m << "%), додається " << keys << " нових ключів." << endl;
    cout << "Очікувано колізій: " << keys * fill / m << " з " << keys
         << " (зайнято " << fill*100/m << "% комірок)." << endl << endl;

    for (int r = 0; r < 3; r++) {
        mt19937 gen(1000 + r);
        vector<uint32_t> base = makeKeys(ranges[r], fill, gen);
        vector<uint32_t> test = makeKeys(ranges[r], keys, gen);

        cout << "Діапазон ключів " << rname[r] << ":" << endl;
        for (int f = 0; f < 3; f++) {
            vector<char> table(m, 0);
            for (int i = 0; i < fill; i++) {
```

```
                uint32_t h = (f == 0) ? shabal512of(base[i]) : myHashOf(base[i]);
                int idx = (f == 2) ? (int)divHash(h, m) : (int)(h >> (32 - bits));
                table[idx] = 1;
            }
            int hit = 0;
            for (int i = 0; i < keys; i++) {
                uint32_t h = (f == 0) ? shabal512of(test[i]) : myHashOf(test[i]);
                int idx = (f == 2) ? (int)divHash(h, m) : (int)(h >> (32 - bits));
                if (table[idx]) hit++;
                else table[idx] = 1;
            }
            cout << "    " << names[f] << ": " << hit << " колізій ("
                 << hit*1000/keys << "‰)" << endl;
        }
        cout << endl;
    }
}




bool selftest() {
    uint32_t h[16], ref[16] = {0xff5d2dfc, 0xf6b7705d, 0xfcc2f8b1, 0xfef9c1c8,
                               0x42e53499, 0x0ceded57, 0xa239b5f2, 0xcc190aef,
                               0x8d4fa8ff, 0xe435a19f, 0xf5093cbd, 0x27a9f390,
                               0xac03d6eb, 0x9e72eb29, 0xf09a2a6f, 0xc68dad31};
    shabal512(vector<unsigned char>(), h);
    for (int i = 0; i < 16; i++)
        if (h[i] != ref[i]) return false;
    return true;
```

```
}



int main() {

    string file = "getfile.pdf";

    int choice = -1;



    cout << "Лабораторна робота №5. Хеш-функції та хеш-таблиці." << endl;

    cout << "Варіант 1.12 — SHABAL, застосування 3б), власна функція методом множення." <<
endl;

    cout << "Перевірка SHABAL-512 за еталоном: " << (selftest() ? "УСПІШНО" : "ПОМИЛКА") <<
endl;

    cout << "Файл для обробки (Enter — " << file << "): ";

    getline(cin, file);

    if (file.empty()) file = "getfile.pdf";



    while (choice != 0) {

        cout << "\n1. Контрольна сума файлу\n2. Дописати суму в кінець файлу\n"

                "3. Перевірити дописану суму\n4. Порівняння хеш-функцій\n0. Вихід\n"

                "Вибір: ";

        if (!(cin >> choice)) break;

        cin.ignore(1000, '\n');



        switch (choice) {

            case 1: calcChecksum(file); break;

            case 2: appendChecksum(file); break;

            case 3: verifyChecksum(file); break;

            case 4: collisions(); break;

            case 0: break;

            default: cout << "Введіть число від 0 до 4." << endl;

        }
```

```
    }

    cout << "Кінець" << endl;

    return 0;

}
```

РЕЗУЛЬТАТИ ТЕСТУВАННЯ ТА РОБОТИ ПРОГРАМИ

Коректність реалізації SHABAL-512 перевірено двома способами. По-перше, функція звірена з еталонною реалізацією sph_shabal.c на 171 тестовому векторі (довжини повідомлень від 0 до 5000 байт) — розбіжностей немає. По-друге, при запуску програма звіряє хеш порожнього повідомлення із еталонним значенням Shabal-512("") = ff5d2dfc f6b7705d fcc2f8b1 fef9c1c8 42e53499 0ceded57 a239b5f2 cc190aef 8d4fa8ff e435a19f f5093cbd 27a9f390 ac03d6eb 9e72eb29 f09a2a6f c68dad31.

Вивід програми:

```
Лабораторна робота №5. Хеш-функції та хеш-таблиці.
Варіант 1.12 — SHABAL, застосування 3б), власна функція методом множення.
Перевірка SHABAL-512 за еталоном: УСПІШНО
Файл для обробки (Enter — getfile.pdf):
1. Контрольна сума файлу
2. Дописати суму в кінець файлу
3. Перевірити дописану суму
4. Порівняння хеш-функцій
0. Вихід
Вибір: Файл: test_copy.pdf   (432484 байт)
SHABAL-512 (512 біт):
aa219a5db5a98d7020921c91608ccf559f4e41d15d58ddcb13af014be2eaea3bb673097f455e52895c9db
6aff0f72dc2ae32144705e300e0c5f5038ff83277de
Власна, метод множення (32 біт): 760ac2fa
Той самий хеш за методом дішення: 762

1. Контрольна сума файлу
2. Дописати суму в кінець файлу
3. Перевірити дописану суму
4. Порівняння хеш-функцій
0. Вихід
Вибір: Контрольну суму 64 байти дописано в кінець файлу test_copy.pdf

1. Контрольна сума файлу
2. Дописати суму в кінець файлу
3. Перевірити дописану суму
4. Порівняння хеш-функцій
0. Вихід
Вибір: Перевірка УСПІШНА — контрольна сума збігається.

1. Контрольна сума файлу
2. Дописати суму в кінець файлу
3. Перевірити дописану суму
4. Порівняння хеш-функцій
0. Вихід
Вибір: Розмір таблиці 8192 комірок, заповнення 4915 (59%), додається 700 нових
ключів.
Очікувано колізій: 419 з 700 (зайнято 59% комірок).

Діапазон ключів 0..10^4:
    SHABAL-512: 515 колізій (735‰)
    Множення: 534 колізій (762‰)
    Дішення: 413 колізій (590‰)
```

```
42.
43. Діапазон ключів 0..10^6:
44.     SHABAL-512: 336 колізій (480%)
45.     Множення: 325 колізій (464%)
46.     Дішення: 336 колізій (480%)
47.
48. Діапазон ключів 0..10^9:
49.     SHABAL-512: 334 колізій (477%)
50.     Множення: 318 колізій (454%)
51.     Дішення: 330 колізій (471%)
52.
53.
54. 1. Контрольна сума файлу
55. 2. Дописати суму в кінець файлу
56. 3. Перевірити дописану суму
57. 4. Порівняння хеш-функцій
58. 0. Вихід
59. Вибір: Кінець
60.
```

БЛОК-СХЕМА ФУНКЦІЇ СТИСНЕННЯ SHABAL (ГОСТ)

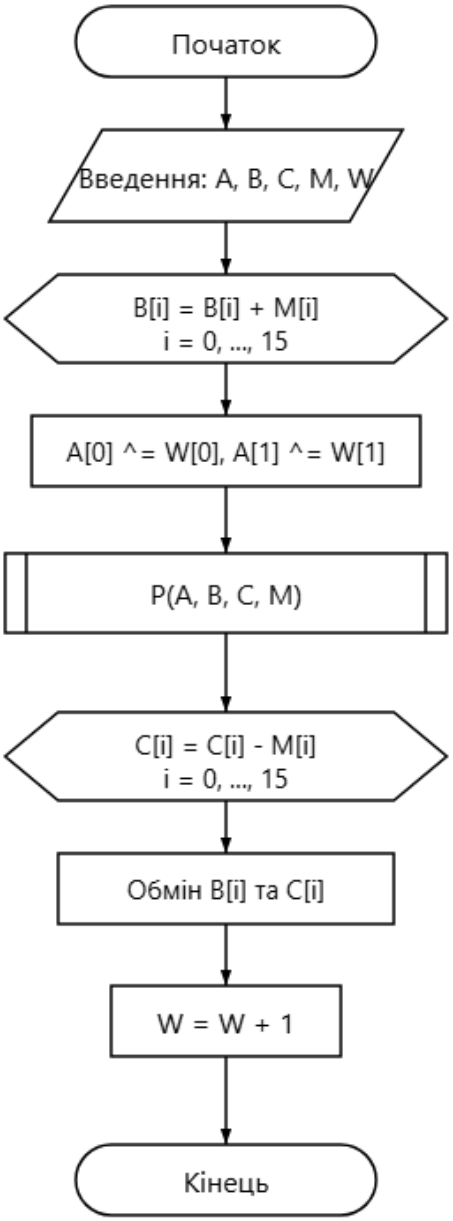


Рисунок 1 — Функція стиснення одного блоку Shabal

БЛОК-СХЕМА ПЕРЕВІРКИ КОНТРОЛЬНОЇ СУМИ (ГОСТ)

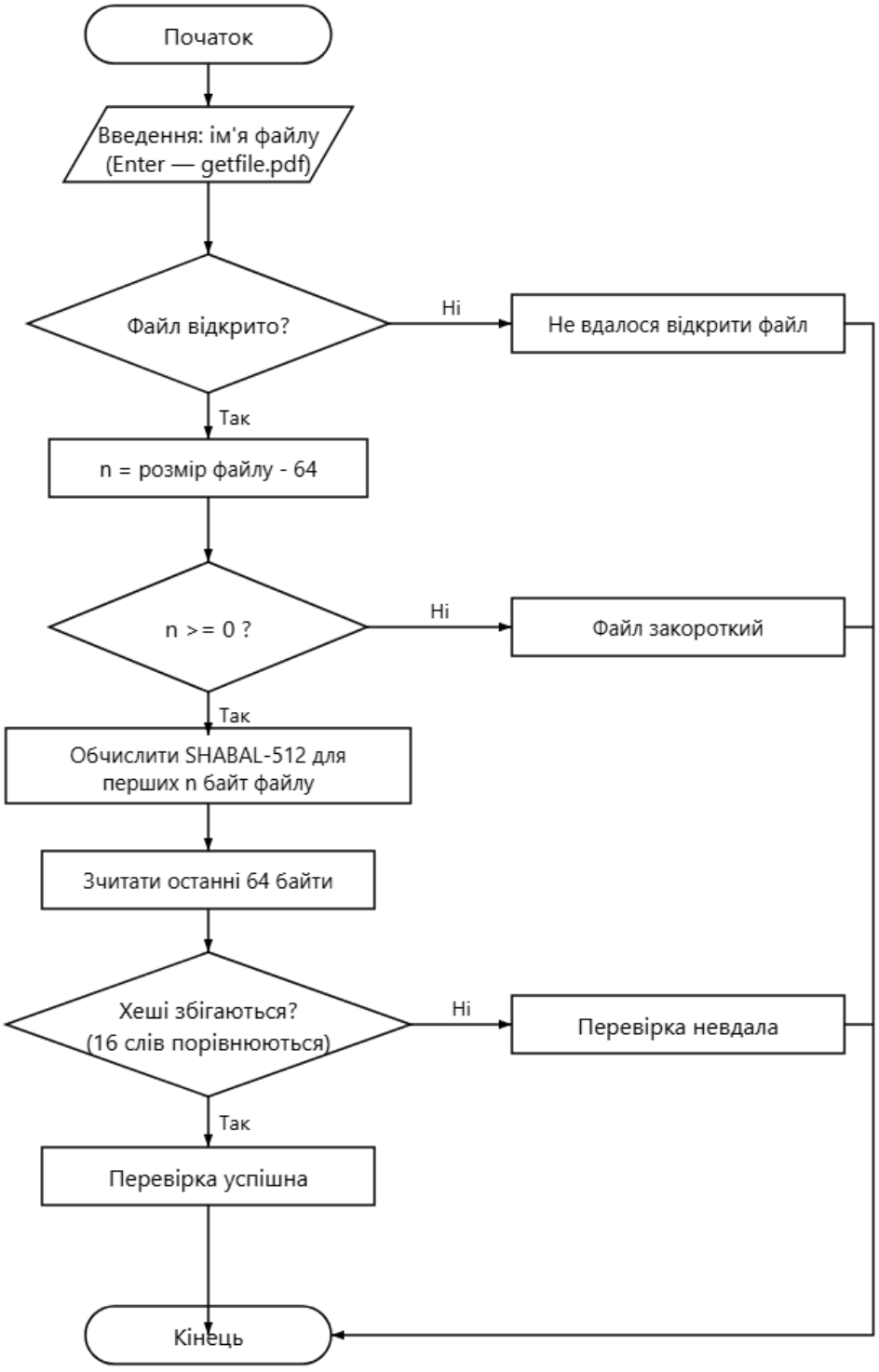


Рисунок 2 — Дописування та перевірка контрольної суми файлу

Вінниця 2026

Висновок: У результаті виконання лабораторної роботи реалізовано хеш-функцію SHABAL-512 та власну хеш-функцію методом множення. Реалізація SHABAL побудована згідно зі специфікацією (перестановка P з 48 раундами, нелінійні множення $U(x)=3x$ і $V(x)=5x$, фіналізація з трьома додатковими раундами) і перевірена на 171 тестовому векторі проти еталонної реалізації — усі значення збігаються. Застосування 3б) реалізовано повністю: програма обчислює контрольну суму файлу, дописує її 64 байти в кінець файлу, а окремий пункт меню повторно обчислює хеш від частини файлу без суми та порівнює її із дописаними байтами; перевірка успішна, отже механізм контролю цілісності працює коректно. Експеримент з колізіями показав, що при таблиці на 8192 комірок, заповненій на 59 %, очікувано близько 419 колізій з 700 нових ключів. Усі три хеш-функції дали близькі результати: SHABAL-512 — 334–515, метод множення — 318–534, метод дішення — 330–413 колізій. Найважливіший висновок експерименту: кількість колізій визначається насамперед діапазоном вхідних даних відносно розміру таблиці, а не вибором хеш-функції. Для вузького діапазону $0..10^4$ при таблиці 8192 комірок усі функції дали в 1,5 раза більше колізій (735–762 ‰), ніж для широких діапазонів $0..10^6$ та $0..10^9$ (454–480 ‰). Реалізація використовує лише стандартні засоби мови C++, без сторонніх бібліотек.

Вінниця 2026